MECHATRONIC SYSTEM INTEGRATION (MCTA 3203)
SEMESTER 1, 2026/2027

EXPERIMENT 2 : Serial Communication

SECTION 1 | GROUP 2

SUBMISSION DATE : 18 OCTOBER 2026

NO.	MATRIC NO.	NAME
1	2413891	AHMAD ZULQARNAIN BIN ABDULLAH
2	2412721	AHMAD FAUZAN BIN HARMAN
3	2410299	AHMAD SOLIHIN BIN MOHAMMAD NAZWAN
4	2417389	AIMAN HASIF BIN MOHAMAD HISYAM

ABSTRACT

This activity focuses on serial communication between an Arduino and a computer using Phyton.

The main objective is to understand how data can be sent and received between both devices to control and monitor electronic components.

The activity involves reading potentiometer values, controlling a servo motor, and using an LED as an indicator. Arduino is used to read input values and control the hardware, while Phyton is used to send commands and display the data on the computer.

The potentiometer is used to adjust the input values, which can affect the servo motor's position and the LED's operation. In addition, matplotlib is used to display the data in a real time graph. This project reflects how to integrate hardware and software in a simple mechatronics system.

TABLE OF CONTENTS

ABSTRACT **1**
TABLE OF CONTENTS **2**
PART 1 : Reading Potentiometer Values via Serial Communication **2**
1.0 INTRODUCTION & THEORY 3
1.0.1 Introduction 3
1.0.2 Theory 3
1.1 MATERIALS AND EQUIPMENT 4
1.2 EXPERIMENTAL SETUP 4
1.2.1 Circuit Setup: 4
1.2.2 Circuit diagram: 5
1.3 METHODOLOGY & PROGRAMMING LOGIC 6
1.4 DATA COLLECTION & ANALYSIS 7
Data Collection 7
Data Analysis 8
1.5 RESULT & DISCUSSION 8
1.5.1 Result 8
1.5.2 Discussion 8
1.6 CONCLUSION 9
PART 2 : Controlling Servo Motor via Serial Communication **10**
1.0 INTRODUCTION & THEORY 10
1.0.1 Introduction 10
1.0.2 Theory 10
1.1 MATERIALS AND EQUIPMENT 11
1.2 EXPERIMENTAL SETUP 12
1.2.1 Circuit Setup: 12
1.2.2 Circuit diagram: 12
1.3 METHODOLOGY & PROGRAMMING LOGIC 13
1.4 DATA COLLECTION & ANALYSIS 14
1.4.1 Data Collection 14
1.4.2 Data Analysis 15
1.5 RESULT & DISCUSSION 15
1.5.1 Result 16
1.5.2 Discussion 16
1.6 CONCLUSION 16
RECOMMENDATIONS **17**
REFERENCES **18**
APPENDICES **19**
ACKNOWLEDGEMENT **24**
STUDENT DECLARATION **24**

PART 1 : Reading Potentiometer Values via Serial Communication

1.0 INTRODUCTION & THEORY

1.0.1 Introduction

This project demonstrates a hardware-software integration between an Arduino microcontroller and a Python script running on a computer. By leveraging universal asynchronous receiver-transmitter (UART) serial communication, the system reads real-time analog sensor data from a potentiometer connected to the Arduino, transmits it to Python for processing, and executes a closed-loop control action by sending commands back to turn an LED ON or OFF based on a specific threshold.

1.0.2 Theory

- **Analog-to-Digital Conversion (ADC):** The Arduino features an internal 10-bit Analog-to-Digital Converter. When connected to a potentiometer, the analog voltage divider circuit varies the voltage at analog pin A0 from $0V$ to $5V$. The ADC maps this voltage range to discrete digital values ranging from 0 to 1023.
- **Serial Communication (UART):** Communication between the Arduino and the computer is established over a serial interface at a standard baud rate (e.g., **9600 bps**). Data streams asynchronously in both directions:
    - *Arduino to PC:* The Arduino continuously transmits sensor readings using Serial.println().
    - *PC to Arduino:* The Python script evaluates the incoming readings and writes control bytes (b'H' or b'L') back to the serial buffer.
- **Closed-Loop Control Logic:** The Python script acts as the master controller. It monitors the incoming integer stream, compares each reading against a midpoint threshold (half of 1023, which is approximately 512), and triggers a corresponding digital output state on the Arduino.

1.1 MATERIALS AND EQUIPMENT

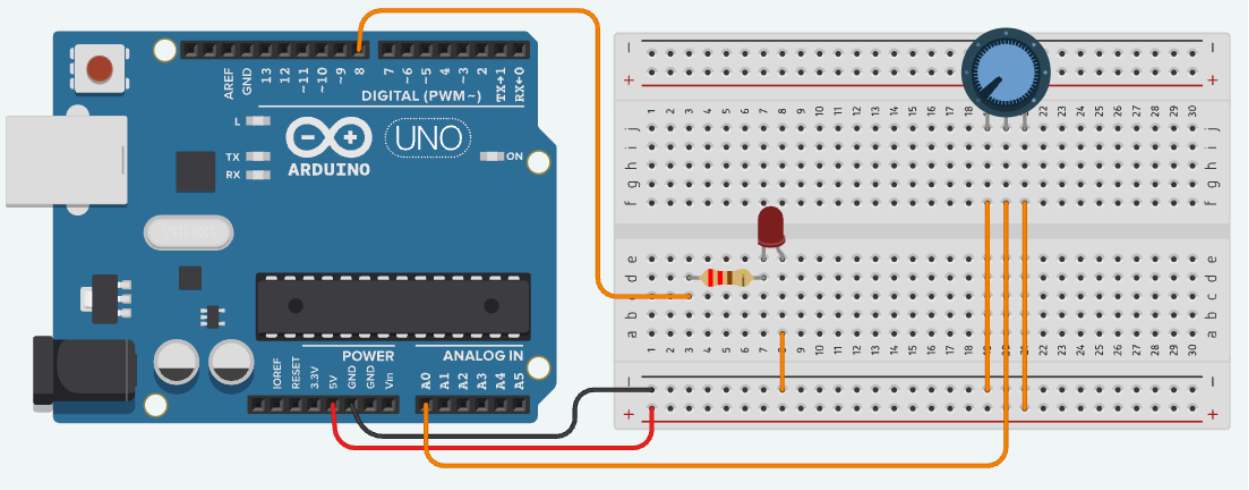
1. Arduino board
2. Potentiometer
3. LED
4. 220 Ω resistor
5. Jumper wires
6. Breadboard

1.2 EXPERIMENTAL SETUP

1.2.1 Circuit Setup:

1. The potentiometer legs from the left were connected to the ground, A0, and 5V.
2. The cathode of a red LED in series with a 220Ω resistor was connected to digital pin 8 of the Arduino.
3. The other leg of the LED was connected to the ground.

1.2.2 Circuit diagram:



1.3 METHODOLOGY & PROGRAMMING LOGIC

Experiment steps:

1. The Arduino was connected to the computer via a USB cable.
2. The sketch was uploaded to the Arduino using the Arduino IDE.
3. The Python script was run on the computer.
4. As the potentiometer knob was turned, the potentiometer readings were displayed in the Python terminal.
5. The Serial Plotter was opened: In the Arduino IDE, Tools -> Serial Plotter was selected.
6. The correct COM port was selected: In the Serial Plotter, the correct COM port to which the Arduino was connected was chosen.
7. The baud rate was set: It was ensured that the baud rate in the Serial Plotter matched the one set in the Arduino code (e.g., 9600).
8. Real-time data was read: As the potentiometer knob was turned, the potentiometer readings were displayed in real-time by the Serial Plotter, creating a graphical representation of the data. The changes in the values were observed as the potentiometer was adjusted.
9. The Plotter was customized: Settings such as the graph range, labels, and colors were adjusted.
10. The circuit was extended by adding an LED in series with a 220Ω resistor. The Python code was updated so that the LED was turned ON automatically when the potentiometer value exceeded half of its maximum range, and was turned OFF otherwise.

*IDE and Python code in appendices.

1.4 DATA COLLECTION & ANALYSIS

Data Collection

State / Position	Raw ADC Reading (0−1023)	Threshold Condition (>512)	Command Sent by Python	LED Status
Minimum (Fully Counter-Clockwise)	0	False	b'L'	OFF
Quarter Turn	256	False	b'L'	OFF
Midpoint (Threshold Boundary)	512	False	b'L'	OFF
Three-Quarter Turn	768	True	b'H'	ON
Maximum (Fully Clockwise)	1023	True	b'H'	ON

Table 1

Data Analysis

The data confirms a linear correlation between the physical rotation of the potentiometer knob and the digital ADC values produced by the Arduino. The Python script successfully intercepts the data stream in real-time (ser.in_waiting), filters non-digit lines, and reliably executes the binary decision logic without significant latency.

1.5 RESULT & DISCUSSION

1.5.1 Result

Terminal Output: The Python terminal successfully printed real-time updates of the potentiometer value (e.g., Potentiometer Value: 768).
Actuator Response: The LED responded instantly when the potentiometer crossed the 512 threshold, illuminating when turned past the halfway mark and extinguishing when lowered below it.
Exclusivity of Serial Port: Attempting to open the Arduino IDE Serial Plotter while the Python script was running resulted in a permission error or blocked communication, confirming that the serial port can only be actively bound by one application at a time.

1.5.2 Discussion

Advantages: This architecture combines the robust low-level signal acquisition capabilities of microcontrollers with the advanced computational, analytical, and graphical libraries available in Python (such as Matplotlib or Pandas).
Design Considerations & Limitations:

- *Polling Rate:* The $100ms$ delay (delay(100) in Arduino and timeout=1 in Python) ensures stable communication but limits high-frequency response. Removing or reducing delays requires careful optimization to prevent buffer overflows.
- *Port Management:* Users must ensure that the correct COM port is configured and that debugging tools like the Arduino Serial Monitor or Plotter are closed before launching the Python script.

1.6 CONCLUSION

The project successfully demonstrated bidirectional serial communication and threshold-based control between an Arduino and a Python environment. By reading analog inputs, transmitting them over UART, processing the data via a Python script, and returning automated control signals, the system established a functional hardware-in-the-loop control loop. This foundation can be extended to more advanced applications, such as real-time data logging to CSV files, live graphical plotting using Python libraries, or implementing PID control loops for automated actuators.

PART 2 : Controlling Servo Motor via Serial Communication

1.0 INTRODUCTION & THEORY

1.0.1 Introduction

This project builds upon basic serial communication by integrating closed-loop actuator control, multi-state machine execution, and real-time data visualization. A potentiometer is used to dynamically adjust the position of a servo motor via an Arduino microcontroller. Simultaneously, an LED provides physical feedback based on the servo angle, and a Python script running Matplotlib visualizes the incoming data stream in real time while supporting remote execution halting commands.

1.0.2 Theory

- **Pulse Width Modulation (PWM) & Servo Control:** Hobby servo motors are controlled using PWM signals, where the duration of a positive pulse (typically between $1ms$ and $2ms$) determines the angular position of the shaft (0° to 180°). The Arduino's Servo library abstracts these pulse calculations.
- **Data Mapping:** The analog input from the potentiometer (0 to 1023) is linearly scaled to match the servo angle range (0° to 180°) using the mapping function:

    - $Angle = map(potValue, 0, 1023, 0, 180)$

- **Bidirectional Control States:** The Arduino code utilizes a boolean state flag (isRunning) managed via UART communication. Sending an ASCII character ('x' / 'X') pauses the servo and deactivates the indicator LED, whereas 'r' / 'R' resumes normal operation.
- **Real-Time Graphical Plotting:** The Python script uses matplotlib in interactive mode (plt.ion()) with a rolling window (last 50 points) to dynamically update and render time-series data without blocking serial reception.

1.1 MATERIALS AND EQUIPMENT

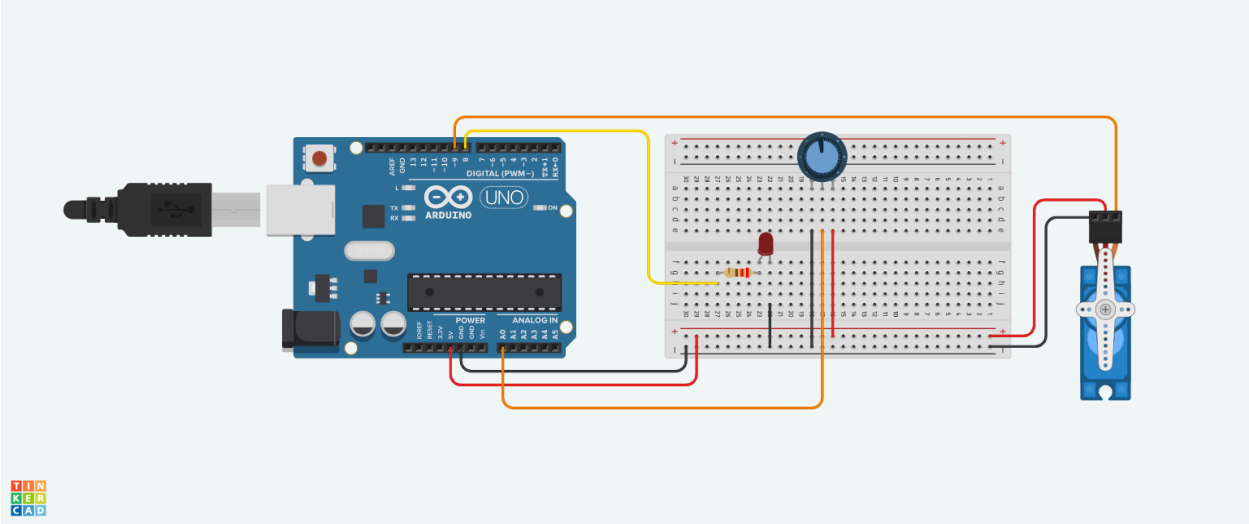
1. Arduino Board + USB cable
2. Servo Motor
3. Jumper Wires
4. Potentiometer
5. LED
6. Breadboard
7. Resistor

1.2 EXPERIMENTAL SETUP

1.2.1 Circuit Setup:

1. The servo's motor signal wire was connected to digital pin D9, VCC to 5V and GND to GND on the Arduino Uno.
2. The potentiometer's outer terminals were connected to 5V and GND, middle terminal to analog pin A0.
3. The LED's anode (long leg) was connected to digital pin D8 through a current limiting resistor and its cathode (short leg) to GND.

1.2.2 Circuit diagram:



1.3 METHODOLOGY & PROGRAMMING LOGIC

Experiment steps:

1. The circuit was assembled by connecting the servo motor, potentiometer and LED to the Arduino Uno according to the circuit diagram.
2. The Arduino Uno was connected to the computer using a USB cable.
3. The Arduino code was written and uploaded to the board via IDE.
4. The required Python libraries were installed, including PySerial and Matplotlib.
5. The correct COM port was configured by opening the Python and baud rate of 9600.
6. The script was run to establish serial communication between the Arduino and computer.
7. The potentiometer was rotated gradually and observed the corresponding changes in the servo motor's position.
8. The LED response was observed according to the programmed condition.
9. The potentiometer readings were monitored using PyCharm and observed their changes on the real-time graph.
10. The observations were recorded and the PyCharm was stopped after completing the experiment.

*IDE code in appendices.

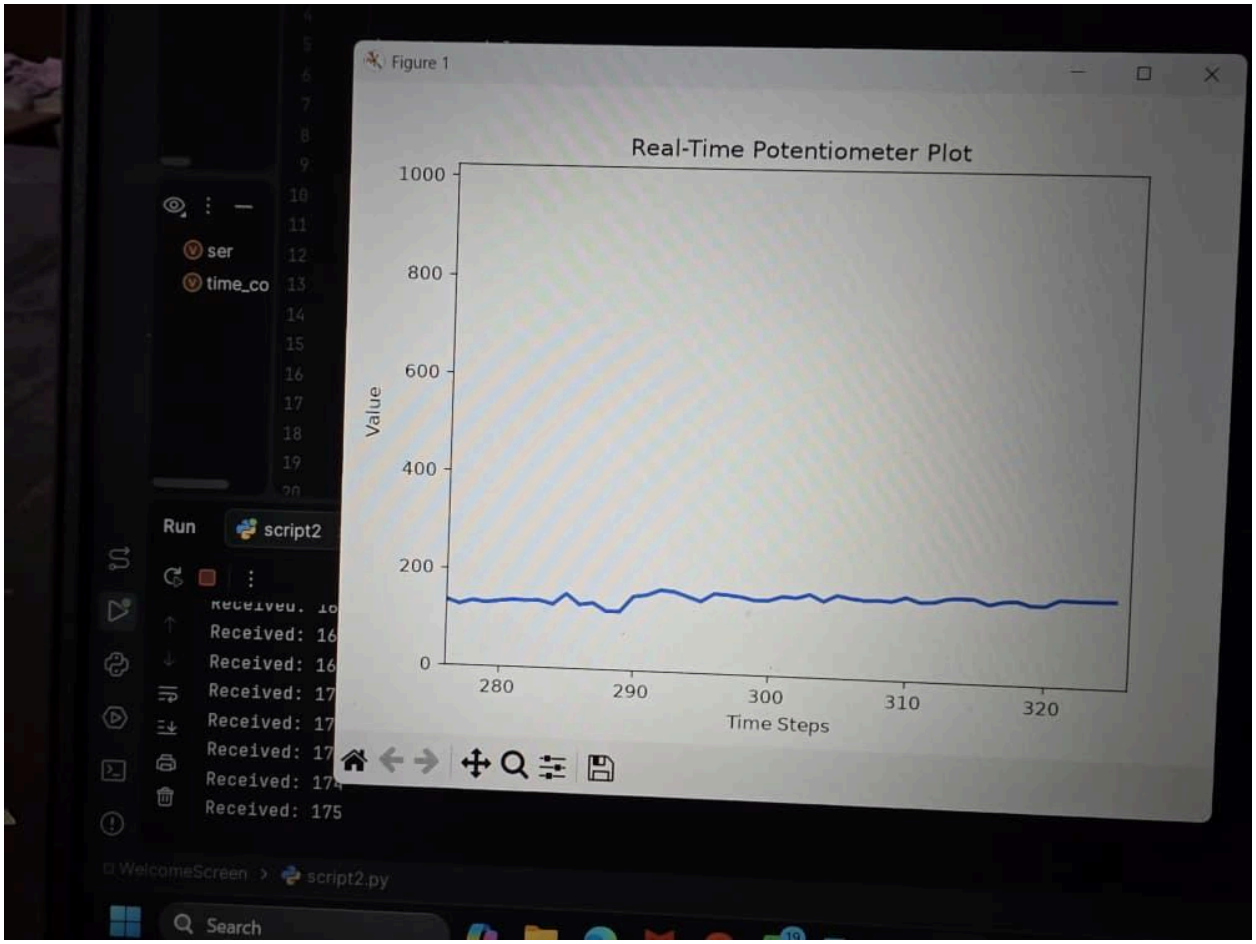
1.4 DATA COLLECTION & ANALYSIS

1.4.1 Data Collection

Data were collected by monitoring the potentiometer readings changes, servo motor angles and LED responses during the experiment.
The potentiometer generated analog values ranging from 0 to 1023. These values are then converted into servo angles between 0° and 180°.
Table 2 below were the expected values between potentiometer readings and servo angles.

Potentiometer Reading	Approximate Servo Angle	Servo Position
0	0°	Minimum
256	45°	Lower angular
512	90°	Middle
768	135°	Upper angular
1023	180°	Maximum

Table 2



However, based on the graph, the potentiometer readings fluctuated approximately between 110 and 150, with most readings around 120-140. The graph displayed the readings against successive time steps, allowing changes in the potentiometer input to be observed.

1.4.2 Data Analysis

The graph shows that the potentiometer readings fluctuated slightly throughout the experiment. The readings increased and decreased as the input values changed, demonstrating that the Arduino successfully collected and transmitted data to Python.
Most readings remained relatively stable, with small variations caused by potentiometer adjustment.
Overall, the real-time graph demonstrated that serial communication and graphical visualisation were functioning, allowing the potentiometer readings to be monitored continuously.

1.5 RESULT & DISCUSSION

1.5.1 Result

The experiment demonstrated the principle of controlling a servo motor using electronic input and serial communication. The potentiometer provided a variable analog input, which the Arduino processed to determine the servo motor's angular position.
As the potentiometer was rotated, its analog readings changed, causing the servo motor to move to different angles. The Python successfully displayed the potentiometer readings and visualised the changes using a real time graph.

1.5.2 Discussion

The experiment demonstrated how the Arduino reads potentiometer values and controls a servo motor. Serial communication allowed the Arduino to transmit data to Python for monitoring and graphical visualization, also the LED provided additional visual feedback. However, small fluctuations in readings occurred due to electrical noise.

1.6 CONCLUSION

The experiment was successfully demonstrated on how to control a servo motor using an Arduino Uno, a potentiometer and serial communication with a computer.

The servo motor's position changed according to the potentiometer input, while Python script enabled the real-time data monitoring.

This experiment enhanced the understanding of serial communication and actuator control in mechatronic systems.

RECOMMENDATIONS

Throughout the experiment, we found a lot of improvement needed in order for the better result of display and output.

To ensure it is working better for Task 1, things such:

1. Check the circuit connections such as the potentiometer, LED, and resistor are connected correctly to the Arduino to avoid circuit problems.
2. Make sure the baud rate in the Arduino code and Phyton script is the same and close the Arduino Serial Plotter before running Phyton.
3. Adjust the potentiometer and check that the LED turns ON when the potentiometer value exceeds half of its maximum range and turns OFF otherwise.

And for Task 2:

1. Applying a simple moving-average filter to improve the potentiometer signal stability that can reduce the fluctuations in analog readings.
2. Using a consistent data format, such as transmitting the potentiometer reading and servo angle as separated value to improve the serial communication.
3. Repeating the measurement that can evaluate the consistency by recording the readings.

REFERENCES

1. Servo motor and Serial. (2022, November 18). Arduino Forum. https://forum.arduino.cc/t/servo-motor-and-serial/1055110
2. Using serial monitor to control servo motor. (n.d.). Arduino Project Hub. https://projecthub.arduino.cc/Kub_Luk/using-serial-monitor-to-control-servo-motor-c55083
3. Dahl, Ø. N. (2023, November 28). *Arduino Potentiometer – Circuit and code example*. Build Electronic Circuits. https://www.build-electronic-circuits.com/arduino-potentiometer/
4. Control a servo with a potentiometer! (n.d.). Arduino Project Hub. https://projecthub.arduino.cc/vansteenkistejoren/control-a-servo-with-a-potentiometer-c13c16
5. Autodesk. (n.g.). *Tinkercad Circuits* [Computer software]. https://www.tinkercad.com

APPENDICES

Task 1:

Arduino

```
const int potPin = A0;
const int ledPin = 8;

void setup() {
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  // Read potentiometer and send to Python
  int potValue = analogRead(potPin);
  Serial.println(potValue);

  // Check if Python sent a command back
  if (Serial.available() > 0) {
    char command = Serial.read();
    if (command == 'H') {
      digitalWrite(ledPin, HIGH); // Turn LED ON
    } else if (command == 'L') {
      digitalWrite(ledPin, LOW);  // Turn LED OFF
    }
  }

  delay(100); // Faster response time for control loop
}
```

Pycharm

```
import serial
import time

ser = serial.Serial('COM3', 9600, timeout=1)
time.sleep(2) # Allow connection to settle

try:
  while True:
    if ser.in_waiting > 0:
      line = ser.readline().decode('utf-8').strip()
      if line.isdigit():
        pot_value = int(line)
        print(f"Potentiometer Value: {pot_value}")

        # Threshold check: Half of 1023 is ~512
        if pot_value > 512:
          ser.write(b'H')  # Send command to turn LED ON
        else:
          ser.write(b'L')  # Send command to turn LED OFF

except KeyboardInterrupt:
  ser.close()
  print("Serial connection closed.")
```

Task 2:
Arduino

```
#include <Servo.h>

Servo myServo;
const int potPin = A0;
const int servoPin = 9;
const int ledPin = 8;
bool isRunning = true;

void setup() {
  Serial.begin(9600);
  myServo.attach(servoPin);
  pinMode(ledPin, OUTPUT);
}

void loop() {

  if (Serial.available() > 0) {
    char cmd = Serial.read();
    if (cmd == 'x' || cmd == 'X') {
      isRunning = false;
    } else if (cmd == 'r' || cmd == 'R') {
      isRunning = true;
    }
  }

  if (isRunning) {
    int potValue = analogRead(potPin);
    // Map potentiometer (0-1023) to servo angle (0-180 degrees)
    int angle = map(potValue, 0, 1023, 0, 180);

    myServo.write(angle);

    // LED feedback: turns ON if angle exceeds 90 degrees, turns OFF otherwise
    if (angle > 90) {
      digitalWrite(ledPin, HIGH);
    } else {
      digitalWrite(ledPin, LOW);
    }
```

```
    // Send the angle back to Python over serial
    Serial.println(angle);
  } else {
    // Halted state: turn off LED and keep servo still
    digitalWrite(ledPin, LOW);
  }

  delay(50); // Short delay for responsive control
}
```

pycharm

```
import matplotlib

matplotlib.use('TkAgg')

import serial
import matplotlib.pyplot as plt

# Replace 'COM3' with your actual Arduino port
ser = serial.Serial('COM3', 9600, timeout=1)

plt.ion()  # Interactive mode for real-time plot
fig, ax = plt.subplots()
x_vals, y_vals = [], []
time_counter = 0


try:
    while True:
        raw_data = ser.readline()
        pot_value = raw_data.decode('utf-8', errors='ignore').strip()

        if pot_value.isdigit():
            val = int(pot_value)
            print(f"Received: {val}")

            x_vals.append(time_counter)
            y_vals.append(val)
            time_counter += 1

            # Keep a rolling window of the last 50 points
            if len(x_vals) > 50:
                x_vals.pop(0)
                y_vals.pop(0)
```

```
        ax.clear()

        ax.plot(x_vals, y_vals, 'b-', linewidth=2)
        ax.set_ylim(0, 1023)  # Potentiometer range
        ax.set_xlim(min(x_vals), max(x_vals) + 1)
        ax.set_title("Real-Time Potentiometer Plot")
        ax.set_xlabel("Time Steps")
        ax.set_ylabel("Value")

        fig.canvas.draw()
        fig.canvas.flush_events()
        plt.pause(0.05)

except KeyboardInterrupt:
  ser.close()
  plt.ioff()
  plt.show()
  print("Serial connection closed.")
```

ACKNOWLEDGEMENT

We would like to express our deepest appreciation to our instructor and lab personnel for their guidance and support throughout this experiment.

STUDENT DECLARATION

We hereby declare that the work presented in this technical report is entirely our own, except where explicitly acknowledged. We certify that in developing and completing this mechatronics project, we have strictly adhered to the standards of academic integrity and have not engaged in any form of plagiarism or unethical conduct. All sources of information,reference materials, and technical support utilized in this work have been appropriately cited and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that **the original work** is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or person. We hereby certify that this report **has not been done by only one individual** and **all of us have contributed** to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have **read** and **understand** the content of the total report and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore, agreed unanimously that this report shall be submitted for **marking** and this **final printed report** has been **verified by us**.

Signature:



Name: Ahmad Zulqarnain bin Abdullah
Matric Number: 2413891

Read	✔
Understand	✔
Agree	✔

Signature:



Name: Aiman Hasif Bin Mohamad Hisyam
Matric Number: 2417389

Read	✔
Understand	✔
Agree	✔

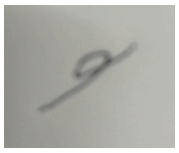
Signature:



Name: Ahmad Fauzan bin Harman
Matric Number: 2412721

Read	✔
Understand	✔
Agree	✔

Signature:



Name: Ahmad Solihin bin Mohammad Nazwan
Matric Number: 2410299

Read	✔
Understand	✔
Agree	✔